

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS

JAVAES

Analizador Léxico para un Lenguaje Personalizado

JAVAES es un lenguaje académico inspirado en Java con palabras clave en español, diseñado para practicar el análisis léxico y sintáctico con ANTLR4 en un front-end de compilador.

Materia	Programación de Sistemas de Base 1
Institución	Universidad Autónoma de Tamaulipas
Semestre	2026-1
Profesor	Muñoz Quintero Dante Adolfo
Repositorio	https://github.com/Falzo186/JavaES.git
Fecha	Mayo 2026

Nombre	Matrícula
Jesus Alejandro Murillo Zavala	2223330180
Adela Vega Ruiz	2223339021
Clemente Pecina Jose Eduardo	2193217022
Saldaña Nieto Pedro Daniel	2223330192

Tabla de Contenido

INDICE

- 1. Introducción 3
- 2. Objetivos 4
 - 2.1 Objetivo General 4
 - 2.2 Objetivos Específicos 4
- 3. Especificación del Lenguaje 5
 - 3.1 Nombre y Propósito..... 5
 - 3.2 Ejemplos Ilustrativos de Código 5
 - 3.3 Catálogo de Tokens 5
- 4. Implementación 7
 - 4.1 Definición de Tokens y Palabras Reservadas — JavaESLexer.g4 7
 - 4.2 Reglas Sintácticas — JavaESParser.g4..... 7
 - 4.3 Lógica del Analizador Léxico — LexerRunner 8
 - 4.4 Manejo y Reporte de Errores — Main.java 8
- 5. Demostración Funcional 10
 - 5.1 Ejecución con Programa Válido..... 10
 - 5.2 Ejecución con Errores Léxicos 10
 - 5.3 Caso con Identificadores Inválidos 11
- 6. Conclusiones 12
- 7. Referencias 13

1. Introducción

El análisis léxico constituye la primera fase de cualquier compilador o intérprete: recibe una secuencia de caracteres como entrada y produce una secuencia de tokens que representan las unidades mínimas con significado del lenguaje. Dominar esta etapa de forma práctica es fundamental en la formación de un ingeniero en sistemas computacionales que desee construir herramientas de procesamiento de lenguajes.

El presente proyecto diseña e implementa un analizador léxico para **JAVAES**, un lenguaje académico inspirado en Java cuyas palabras clave están escritas en español. El nombre refleja su propósito principal: servir como banco de práctica para el análisis léxico y sintáctico mediante ANTLR4, herramienta estándar en el desarrollo de front-ends de compiladores.

El lexer fue implementado en Java usando ANTLR4 como generador de analizadores, con una gramática léxica definida en **JavaESLexer.g4** y una gramática sintáctica en **JavaESParser.g4**. (Aún está en Desarrollo) El sistema transforma archivos fuente *.javaes* en una tabla numerada de tokens (tipo, lexema, línea, columna) e informa con precisión la localización de cualquier error léxico detectado.

Este documento se organiza de la siguiente manera: la Sección 2 presenta los objetivos del proyecto; la Sección 3 describe la especificación del lenguaje JAVAES; la Sección 4 contiene el código fuente comentado de los módulos principales; la Sección 5 muestra capturas de ejecución; finalmente las Secciones 6 y 7 presentan conclusiones y referencias.

2. Objetivos

2.1 Objetivo General

Diseñar un lenguaje personalizado denominado **JVAES** e implementar un analizador léxico funcional en Java con ANTLR4 que transforme programas fuente escritos en dicho lenguaje en listas de tokens, con manejo preciso de errores léxicos que incluya número de línea y columna.

2.2 Objetivos Específicos

1. Definir la especificación informal del lenguaje JVAES mediante ejemplos ilustrativos de código que cubran sus construcciones principales.
2. Establecer un catálogo completo de tokens: palabras reservadas en español, identificadores, números enteros y decimales (incluyendo notación científica, hexadecimal y octal), operadores aritméticos, relacionales y lógicos, delimitadores, cadenas de texto y comentarios.
3. Implementar el analizador léxico usando ANTLR4 con gramáticas .g4, asegurando el reconocimiento correcto de todos los tokens definidos.
4. Implementar la detección y reporte de errores léxicos, indicando número de línea, columna y descripción del problema para cada error encontrado.
5. Probar el lexer con al menos tres programas válidos y dos programas con errores léxicos que cubran los casos descritos en la especificación.
6. Proveer una interfaz de menú interactivo desde consola para facilitar la ejecución y demostración del analizador.

3. Especificación del Lenguaje

3.1 Nombre y Propósito

JAVAES (Java en Español) es un lenguaje imperativo de propósito educativo inspirado en Java, cuyas palabras reservadas han sido sustituidas por equivalentes en español. Está diseñado para que estudiantes hispanohablantes comprendan la estructura de un compilador sin la barrera del idioma en las palabras clave. La extensión de los archivos fuente es **.javaes**.

3.2 Ejemplos Ilustrativos de Código

Ejemplo 1 — Declaración de clase y método:

```
// Ejemplo 1 - Clase y método
clase Calculadora {
    entero sumar(entero a, entero b) {
        retornar a + b;
    }
}
```

Ejemplo 2 — Control de flujo con si/sino:

```
// Ejemplo 2 - Condicional si/sino
entero x = 10;
si (x > 5) {
    imprimir("mayor");
} sino {
    imprimir("menor o igual");
}
```

Ejemplo 3 — Ciclo mientras:

```
// Ejemplo 3 - Ciclo mientras
entero i = 0;
mientras (i < 10) {
    i = i + 1;
}
retornar i;
```

3.3 Catálogo de Tokens

Categoría	Ejemplos	Descripción
KEYWORD	si, sino, mientras, para, clase, retornar, imprimir, leer, verdadero, falso, nulo	Palabras reservadas del lenguaje en español
IDENTIFICADOR	x, total, mi_var, resultado2	Letra o _ seguida de letras, dígitos o _; soporta acentos y ñ
ENTERO	0, 42, 1024	Secuencia de dígitos sin punto decimal
ENTERO_HEX	0x1F, 0xFF	Número en base hexadecimal
ENTERO_OCTAL	017, 075	Número en base octal
DECIMAL	3.14, 0.5, 2.0	Dígitos, punto, dígitos obligatorios
DECIMAL_CIENTIFICO	1.5e10, 3.0E-2	Notación científica con exponente

CADENA	"hola", "valor total"	Cadena entre comillas dobles con secuencias de escape
CHARACTER	'a', '\n'	Carácter entre comillas simples
OPERATOR	+, -, *, /, %, =, ==, !=, >, <, >=, <=, &&, , !	Operadores aritméticos, relacionales y lógicos
DELIMITER	;, () { } [] . : :: ->	Separadores, agrupadores y puntuación
COMMENT	// comentario, /* bloque */	Comentarios de línea y bloque (descartados)
ERROR_LEXICO	\$, @var, 2b	Símbolo no reconocido o identificador inválido

Identificadores válidos: comienzan con letra (a-z, A-Z, incluyendo caracteres acentuados y ñ) o guión bajo (_), seguidos de cero o más letras, dígitos o guiones bajos. **Identificadores inválidos** (generan ERROR_LEXICO): comienzan con dígito (ej. 2b) o contienen caracteres no permitidos (ej. \$var).

4. Implementación

A continuación se presenta el código fuente de los módulos principales del analizador léxico. El código completo se encuentra en el repositorio GitHub indicado en la portada. El proyecto compila con **compilar.bat** y se ejecuta con **ejecutar.bat**.

4.1 Definición de Tokens y Palabras Reservadas — JavaESLexer.g4

Define la gramática léxica completa de JAVAES usando ANTLR4. El orden de las reglas es fundamental: los operadores de dos caracteres (como ==, >=) deben preceder a los de un carácter. De igual forma, DECIMAL_CIENTIFICO precede a DECIMAL y este a ENTERO.

```
lexer grammar JavaESLexer;

// Comentarios y espacios — descartados
LINE_COMMENT      : '//' ~[\r\n]* -> skip;
BLOCK_COMMENT     : '/*' .*? '*/' -> skip;
WS                : [ \t\r\n]+ -> skip;

// Palabras reservadas — tipos
ENTERO_TIPO       : 'entero'; DECIMAL_TIPO : 'decimal';
CADENA_TIPO       : 'cadena'; BOOLEANO_TIPO: 'booleano';

// Control de flujo
SI:'si'; SINO:'sino'; MIENTRAS:'mientras'; RETORNAR:'retornar';

// Literales numéricos (orden obligatorio)
DECIMAL_CIENTIFICO : [0-9]+'.' [0-9]+ ('e'|'E') ('+'|'-')? [0-9]+;
DECIMAL            : [0-9]+'.' [0-9]+;
ENTERO_HEX         : '0x' [0-9a-fA-F]+;
ENTERO             : [0-9]+;

// Identificadores (soporta acentos unicode)
IDENTIFICADOR      : [a-zA-Z_\u00C0-\u017F] [a-zA-Z0-9_\u00C0-\u017F]*;

// Operadores (multicarácter antes que monocarácter)
INCREMENTO:'++;'; IGUALDAD:'=='; DESIGUALDAD:'!=';
MAYOR_IGUAL_QUE:'>='; MENOR_IGUAL_QUE:'<=';
SUMA:'+'; RESTA:'-'; MAYOR_QUE:'>'; MENOR_QUE:'<';

// Error léxico
ERROR_LEXICO : . ;
```

4.2 Reglas Sintácticas — JavaESParser.g4

Define la gramática del parser con jerarquía de producciones. La precedencia de operadores se codifica en la jerarquía de reglas: suma encapsula a producto, que encapsula a atom.

```
parser grammar JavaESParser;
options { tokenVocab=JavaESLexer; }

programa: declaracionGlobal* EOF;
declaracionGlobal: claseDecl | metodoDecl | varGlobalDecl;

claseDecl: CLASE IDENTIFICADOR LLAVE_ABIERTA miembroClase* LLAVE_CERRADA;
metodoDecl: tipo IDENTIFICADOR PARENTESIS_ABIERTO parametros?
           PARENTESIS_CERRADO bloque;
```

```

tipo: ENTERO_TIPO | DECIMAL_TIPO | CADENA_TIPO | BOOLEANO_TIPO | VACIO_TIPO;
bloque: LLAVE_ABIERTA instruccion* LLAVE_CERRADA;

// Expresiones con jerarquía de precedencia
expresion : asignacion;
asignacion : comparacion (ASIGNACION asignacion)?;
comparacion: suma ((IGUALDAD|DESIGUALDAD|MAYOR_QUE|MENOR_QUE) suma)*;
suma      : producto ((SUMA | RESTA) producto)*;
producto  : atom ((MULTIPLICACION | DIVISION | MODULO) atom)*;
atom: PARENTESIS_ABIERTO expresion PARENTESIS_CERRADO
    | ENTERO | DECIMAL | CADENA | VERDADERO | FALSO | IDENTIFICADOR;

```

4.3 Lógica del Analizador Léxico — LexerRunner

La clase LexerRunner invoca el lexer generado por ANTLR4 sobre un archivo fuente .javaes y presenta los tokens en una tabla formateada con columnas TOKEN, LEXEMA, LÍNEA y COLUMNA. Los tokens de tipo ERROR_LEXICO se acumulan para desplegarse en un panel separado al final con su ubicación exacta.

```

// LexerRunner.java — fragmento principal
public class LexerRunner {
    public static void ejecutarLexico(String ruta) throws IOException {
        CharStream input = CharStreams.fromFileName(ruta);
        JavaESLexer lexer = new JavaESLexer(input);
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        tokens.fill();

        List<String[]> filas = new ArrayList<>();
        List<String> errores = new ArrayList<>();

        for (Token token : tokens.getTokens()) {
            if (token.getType() == Token.EOF) continue;
            String tipo = JavaESLexer.VOCABULARY.getSymbolicName(token.getType());
            String lexema = token.getText();
            int linea = token.getLine();
            int col = token.getCharPositionInLine() + 1;
            filas.add(new String[]{tipo, lexema,
                                   String.valueOf(linea),
                                   String.valueOf(col)});

            if ("ERROR_LEXICO".equals(tipo)) {
                errores.add("Error léxico en línea " + linea +
                           ", columna " + col + ": '" + lexema + "'");
            }
        }
        // Mostrar tabla y lista de errores...
    }
}

```

4.4 Manejo y Reporte de Errores — Main.java

El punto de entrada Main.java provee un menú interactivo de consola con seis opciones: análisis léxico, análisis sintáctico, árbol sintáctico, ejemplos válidos, ejemplos con error y salida. Los archivos .javaes se listan dinámicamente desde los directorios de recursos.

```

// Main.java — bucle principal
public static void main(String[] args) throws IOException {
    Scanner sc = new Scanner(System.in);
    while (true) {
        System.out.println("JAVAES - Analizador Sintáctico");
        System.out.println("1. Ejecutar análisis léxico");
        System.out.println("2. Ejecutar análisis sintáctico");
        System.out.println("3. Mostrar árbol sintáctico");
    }
}

```



```

        System.out.println("4. Ejecutar ejemplo válido");
        System.out.println("5. Ejecutar ejemplo con error");
        System.out.println("6. Salir");
        String opcion = sc.nextLine().trim();
        switch (opcion) {
            case "1": LexerRunner.ejecutarLexico(resolverRuta(sc)); break;
            case "4": mostrarSubmenEjemplos("Validos", sc); break;
            case "5": mostrarSubmenEjemplos("Invalidos", sc); break;
            case "6": return;
        }
    }
}

```

5. Demostración Funcional

Las siguientes capturas muestran la ejecución del lexer desde la terminal. Los archivos de prueba se ubican en `resources/examples/Validos/` y `resources/examples/Invalidos/`.

5.1 Ejecución con Programa Válido

Se selecciona la opción **4** del menú principal (Ejecutar ejemplo válido) y luego el archivo **ejemplo_basico.javaes**. El programa contiene declaraciones de variables enteras y una expresión aritmética; no presenta errores léxicos.

```
JAVAES - Analizador Sintáctico
1. Ejecutar análisis léxico
2. Ejecutar análisis sintáctico
3. Mostrar árbol sintáctico
4. Ejecutar ejemplo válido
5. Ejecutar ejemplo con error
6. Salir
Seleccione una opción: 4

Ejemplos Validos
1. ejemplo_basico.javaes
2. ejemplo_clase.javaes
3. ejemplo_control_flujo.javaes
4. ejemplo_expandido.javaes
5. ejemplo_if.javaes
6. ejemplo_metodo.javaes
7. ejemplo_while.javaes
0. Volver al menú principal
Seleccione un ejemplo: 1
```

Captura 5.1a — Menú principal y selección del ejemplo válido ejemplo_basico.javaes.

ANÁLISIS LÉXICO - TOKENS DETECTADOS

TOKEN	LEXEMA	LÍNEA	COLUMNA
ENTERO_TIPO	entero	1	1
IDENTIFICADOR	x	1	8
ASIGNACION	=	1	10
ENTERO	10	1	12
PUNTO_Y_COMA	;	1	14
ENTERO_TIPO	entero	2	1
IDENTIFICADOR	y	2	8
ASIGNACION	=	2	10
ENTERO	20	2	12
PUNTO_Y_COMA	;	2	14
ENTERO_TIPO	entero	3	1
IDENTIFICADOR	z	3	8
ASIGNACION	=	3	10
IDENTIFICADOR	x	3	12
SUMA	+	3	14
IDENTIFICADOR	y	3	16
PUNTO_Y_COMA	;	3	17

Captura 5.1b — Tabla de tokens generada para ejemplo_basico.javaes (sin errores léxicos).

5.2 Ejecución con Errores Léxicos

Se selecciona la opción **5** (Ejecutar ejemplo con error) y el archivo **error_lexico.javaes**. El archivo contiene el símbolo `$`, no reconocido por la gramática de JAVAES. El lexer lo registra como **ERROR_LEXICO** en la tabla de tokens y emite el mensaje de error con línea y columna exactas en el panel inferior.

```
=====
JAVAES - Analizador Sintáctico
=====
1. Ejecutar análisis léxico
2. Ejecutar análisis sintáctico
3. Mostrar árbol sintáctico
4. Ejecutar ejemplo válido
5. Ejecutar ejemplo con error
6. Salir
Seleccione una opción: 5

=====
Ejemplos Invalidos
=====
1. error_lexico.javaes
2. error_lexico_operador.javaes
3. error_sintactico.javaes
0. Volver al menú principal
Seleccione un ejemplo: 1

--- Ejemplo: error_lexico.javaes ---
```

TOKEN	LEXEMA	LÍNEA	COLUMNA
ENTERO_TIPO	entero	1	1
ERROR_LEXICO	\$	1	8
IDENTIFICADOR	x	1	9
ASIGNACION	=	1	11
ENTERO	10	1	13
PUNTO_Y_COMA	;	1	15

```
=====
ERROR LÉXICO DETECTADO
=====
Error léxico: carácter inválido '$' en línea 1, columna 8.
```

Captura 5.2 — Detección de símbolo inválido '\$' en línea 1, columna 8. El resto del programa se tokeniza correctamente.

5.3 Caso con Identificadores Inválidos

Se ejecuta el archivo **error_identificador.javaes**. Contiene el identificador `2b` que comienza con dígito, lo cual es inválido. El lexer emite el error: *"Error léxico: identificador inválido '2b' en línea 2, columna 12"*. El resto de los tokens válidos se procesan sin interrupciones.

```
Ejemplos Invalidos
=====
1. error_identificador.javaes
2. error_lexico.javaes
3. error_lexico_operador.javaes
4. error_sintactico.javaes
0. Volver al menú principal
Seleccione un ejemplo: 1

--- Ejemplo: error_identificador.javaes ---
```

ANÁLISIS LÉXICO - TOKENS DETECTADOS

TOKEN	LEXEMA	LÍNEA	COLUMNA
ENTERO_TIPO	entero	1	1
IDENTIFICADOR	procesar	1	8
PARENTESIS_ABIERTO	(	1	16
ENTERO_TIPO	entero	1	17
IDENTIFICADOR	a	1	24
PARENTESIS_CERRADO	)	1	25
LLAVE_ABIERTA	{	1	27
ENTERO_TIPO	entero	2	5
IDENTIFICADOR_INVALIDO	2b	2	12
ASIGNACION	=	2	15
ENTERO	5	2	17
PUNTO_Y_COMA	;	2	18
RETORNAR	retornar	3	5
IDENTIFICADOR	a	3	14
SUMA	+	3	16
IDENTIFICADOR	b	3	18
PUNTO_Y_COMA	;	3	19
LLAVE_CERRADA	}	4	1

ERROR LÉXICO DETECTADO

```
Error léxico: identificador inválido '2b' en línea 2 columna 12
```

Captura 5.3 — Identificador inválido '2b' detectado en línea 2, columna 12. El lexer continúa procesando el resto del archivo correctamente.

6. Conclusiones

Jesus Alejandro Murillo Zavala. Diseñar la gramática léxica en ANTLR4 me permitió entender en la práctica por qué el orden de las reglas es crítico: cuando coloqué ENTERO antes que DECIMAL_CIENTIFICO, el lexer tokenizaba 1.5e10 en fragmentos separados. Verlo fallar y corregirlo fue la forma más efectiva de interiorizar el concepto de 'longest match' en los lexers.

Adela Vega Ruiz. El mayor reto fue el manejo de errores léxicos sin detener el análisis del resto del programa. Implementar el mecanismo de recuperación —marcar el token como ERROR_LEXICO y continuar— requirió entender el flujo interno de ANTLR4. El resultado fue un módulo de errores mucho más útil para el usuario final.

Clemente Pecina Jose Eduardo. Separar la gramática léxica (JavaESLexer.g4) de la gramática sintáctica (JavaESParser.g4) facilitó enormemente las pruebas: podía modificar una expresión regular y ejecutar inmediatamente los casos de prueba sin tocar el parser. Esa separación de responsabilidades es algo que aplicaré en proyectos futuros.

Saldaña Nieto Pedro Daniel. Implementar la detección del error de 'identificador que comienza con dígito' requirió distinguir entre un NUMBER_INT legítimo y uno seguido inmediatamente de letras. La solución fue revisar el contexto del token dentro del manejador de errores, lo cual me hizo comprender mejor los lookaheads en los analizadores léxicos.

7. Referencias

ANTLR Project. (2024). ANTLR 4 Documentation. Recuperado de <https://www.antlr.org/>

Oracle Corporation. (2024). Java SE 21 Documentation. Recuperado de <https://docs.oracle.com/en/java/javase/21/>